MULTI-DIMENSIONAL ARRAYS

CSC103 PROGRAMMING FUNDAMENTALS

RECAP

- Passing an array to a method
- Returning an array from a method
- Foreach loop & arrays
- Searching arrays
  - Linear search
  - Binary search
- Sorting array
  - Insertion sort
  - Selection sort
  - Bubble sort

TODAY'S LECTURE

- Basics of two-dimensional arrays
- Processing two-dimensional arrays
- Passing two-dimensional arrays to methods
- Multidimensional arrays

INTRODUCTION

- Data in a table or a matrix can be represented using a two-dimensional array
- One dimensional array stores linear collection of elements

Distance Table (in miles)

	Chicago	Boston	New York	Atlanta	Miami	Dallas	Houston
Chicago	0	983	787	714	1375	967	1087
Boston	983	0	214	1102	1763	1723	1842
New York	787	214	0	888	1549	1548	1627
Atlanta	714	1102	888	0	661	781	810
Miami	1375	1763	1549	661	0	1426	1187
Dallas	967	1723	1548	781	1426	0	239
Houston	1087	1842	1627	810	1187	239	0

```
double[][] distances = {
  {0, 983, 787, 714, 1375, 967, 1087},
  {983, 0, 214, 1102, 1763, 1723, 1842},
  {787, 214, 0, 888, 1549, 1548, 1627},
  {714, 1102, 888, 0, 661, 781, 810},
  {1375, 1763, 1549, 661, 0, 1426, 1187},
  {967, 1723, 1548, 781, 1426, 0, 239},
  {1087, 1842, 1627, 810, 1187, 239, 0},
};
```

TWO-DIMENSIONAL ARRAY BASICS

- An element in a two-dimensional array is accessed through a row and column index

DECLARING VARIABLES OF TWO-DIMENSIONAL ARRAYS

- Syntax:

```
elementType[][] arrayRefVar;
```

or

```
elementType arrayRefVar[][]; // Allowed, but not preferred
```

- Example:

```
int[][] matrix;
```

or

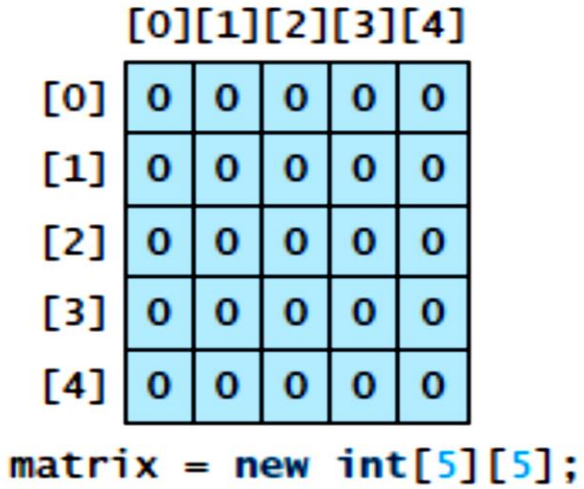
```
int matrix[][]; // This style is allowed, but not preferred
```

CREATING TWO-DIMENSIONAL ARRAYS

- Syntax: `arrRefVar = new elementType[rows][columns];`
- A two-dimensional array of 5-by-5 integer (int) values, assigned to matrix

```
matrix = new int[5][5];
```

- Two subscripts are used in two-dimensional array, one for the row and the other for the column
- Index of each subscript of a two-dimensional array

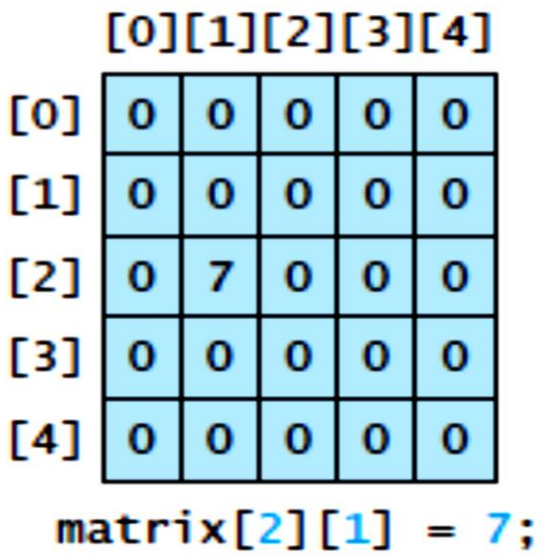


ASSIGNING VALUES TO TWO-DIMENSIONAL ARRAYS

- Syntax:

```
arrRefVar[row_index][column_index] = value;
```

- Example: `matrix[2][1] = 7;`



- A common mistake:

```
matrix[2, 1]
```

```
matrix[0][0] = 1;
matrix[0][1] = 2;
matrix[0][2] = 3;
matrix[0][3] = 4;
matrix[0][4] = 5;
```

	[0]	[1]	[2]	[3]	[4]
[0]	1	2	3	4	5
[1]	0	0	0	0	0
[2]	0	7	0	0	0
[3]	0	0	0	0	0
[4]	0	0	0	0	0

```
matrix[1][0] = 6;
matrix[1][1] = 7;
matrix[1][2] = 8;
matrix[1][3] = 9;
matrix[1][4] = 10;
```

	[0]	[1]	[2]	[3]	[4]
[0]	1	2	3	4	5
[1]	6	7	8	9	10
[2]	0	7	0	0	0
[3]	0	0	0	0	0
[4]	0	0	0	0	0

```
matrix[2][0] = 11;
matrix[2][1] = 12;
matrix[2][2] = 13;
matrix[2][3] = 14;
matrix[2][4] = 15;
```

	[0]	[1]	[2]	[3]	[4]
[0]	1	2	3	4	5
[1]	6	7	8	9	10
[2]	11	12	13	14	15
[3]	0	0	0	0	0
[4]	0	0	0	0	0

ARRAY INITIALIZER

- Array initializer declares, creates, and initializes a two-dimensional array in one statement.

```
int[][] array =
  {1, 2, 3},
  {4, 5, 6},
  {7, 8, 9},
  {10, 11, 12}
};
```

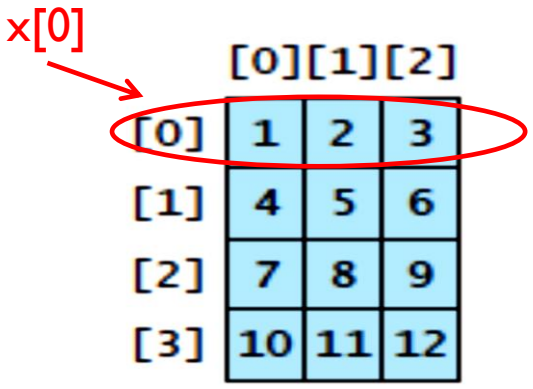
Equivalent

```
int[][] array = new int[4][3];
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```

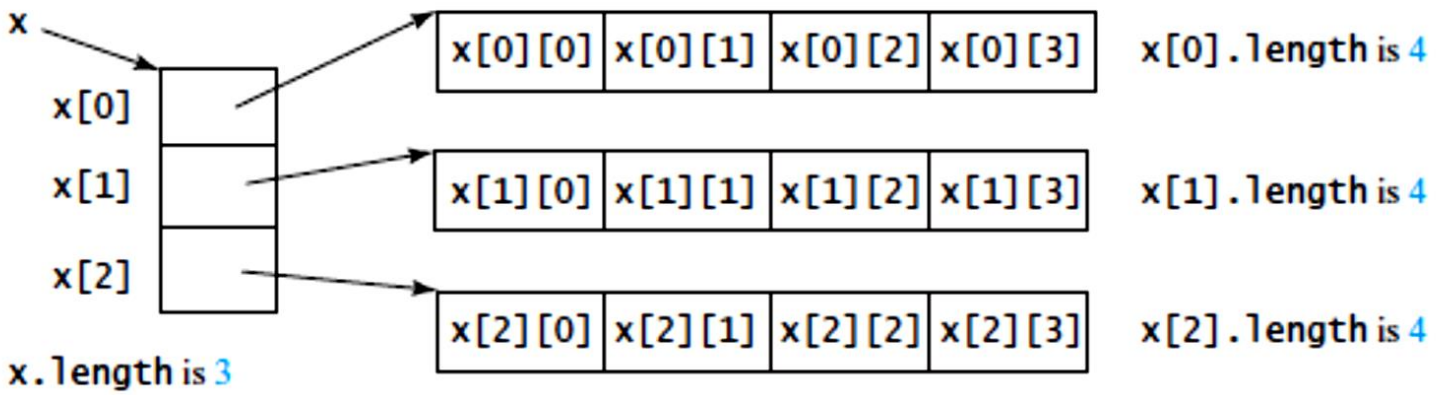
	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9
[3]	10	11	12

OBTAINING LENGTHS OF TWO-DIMENSIONAL ARRAYS

- A two-dimensional array is an array in which each element is a one-dimensional array.
- The length of an array `x` is the number of elements in that array, which is obtained by `x.length`
- `x[0]` represents 1D array at index 0 of x; `x[1]` represents 1D array at index 1 of x; `x[x.length - 1]` represents 1D array at last index of x
- `x[0].length` will give the size of 1D array at index 0 of x; `x[1].length` will give the size of 1D array at index 1 of x, and so on…



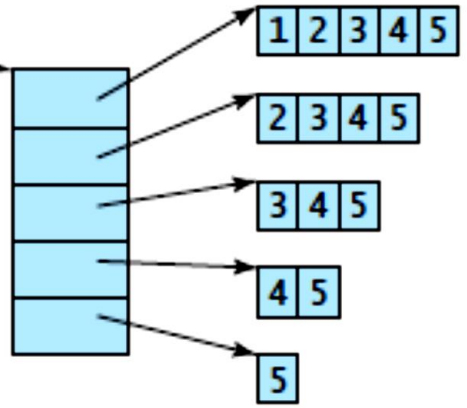
`x.length = 4`
`x[0].length = 3`



RAGGED ARRAYS

- Each row in a 2D array is itself an array
- Rows can have different lengths

```
int[][] triangleArray = {
  {1, 2, 3, 4, 5},
  {2, 3, 4, 5},
  {3, 4, 5},
  {4, 5},
  {5}
};
```



`triangleArray[0].length` is 5
`triangleArray[1].length` is 4
`triangleArray[2].length` is 3
`triangleArray[3].length` is 2
`triangleArray[4].length` is 1

- Creating a ragged array

```
int[][] triangleArray = new int[5][];
triangleArray[0] = new int[5];
triangleArray[1] = new int[4];
triangleArray[2] = new int[3];
triangleArray[3] = new int[2];
triangleArray[4] = new int[1];
```

- Assigning values

```
triangleArray[0][3] = 50;
triangleArray[4][0] = 45;
```

- Incorrect creation

```
int[][] triangleArray = new int[][];
```

PROCESSING 2D ARRAYS

- Nested for loops are often used to process a 2D array
- Processing 2D arrays
  - Initializing arrays with input values
  - Initializing arrays with random values
  - Printing arrays
  - Summing all elements
  - Summing elements by column
  - Row with the largest sum
  - Random shuffling

```
int[][] matrix = new int[3][3];
```

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

row	column	nextInt()

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	0	0	0
[1]	0	0	0
[2]	0	0	0

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

row	column	nextInt()
0	0	10

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	10	0	0
[1]	0	0	0
[2]	0	0	0

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

row	column	nextInt()
0	0	10
	1	20

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	10	20	0
[1]	0	0	0
[2]	0	0	0

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

row	column	nextInt()
0	0	10
	1	20
	2	30

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	10	20	30
[1]	0	0	0
[2]	0	0	0

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

row	column	nextInt()
0	0	10
	1	20
	2	30
1	0	40

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	0	0
[2]	0	0	0

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

row	column	nextInt()
0	0	10
	1	20
	2	30
1	0	40
	1	50

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	0
[2]	0	0	0

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

row	column	nextInt()
0	0	10
	1	20
	2	30
1	0	40
	1	50
	2	60

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	0	0	0

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

row	column	nextInt()
0	0	10
	1	20
	2	30
1	0	40
	1	50
	2	60
2	0	70

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	0	0

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

matrix.length is 3
matrix.length[0] is 3

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	0

row	column	nextInt()
0	0	10
	1	20
	2	30
1	0	40
	1	50
	2	60
2	0	70
	1	80

INITIALIZING ARRAYS WITH INPUT VALUES

```
int[][] matrix = new int[3][3];
```

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
  matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = input.nextInt();
  }
}
```

matrix.length is 3
matrix.length[0] is 3

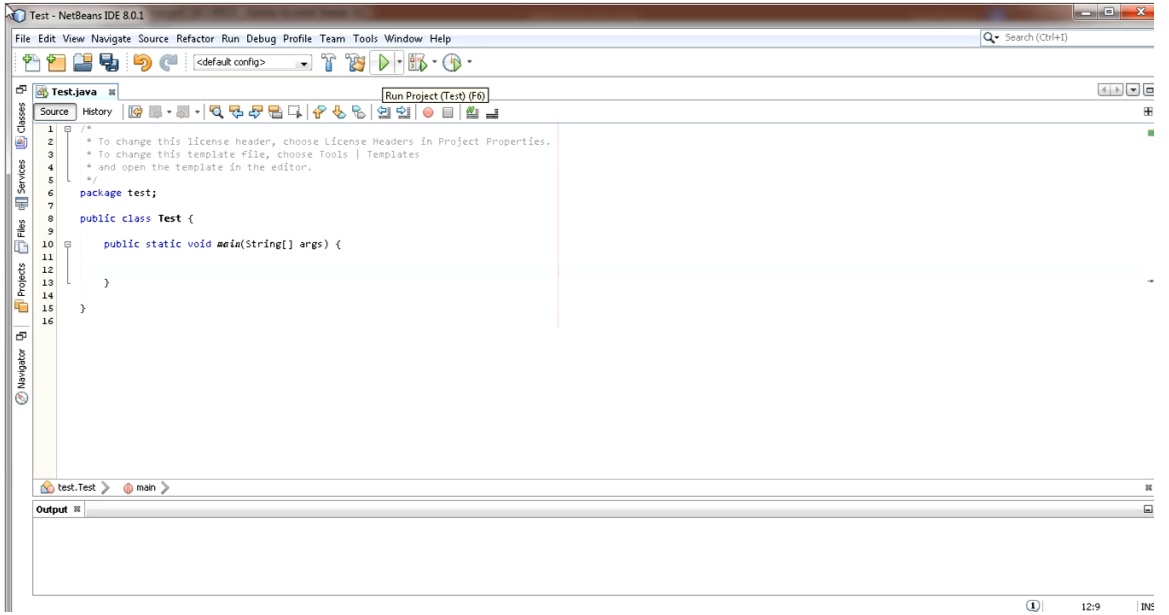
	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	nextInt()
0	0	10
	1	20
	2	30
1	0	40
	1	50
	2	60
2	0	70
	1	80
	2	90

INITIALIZING ARRAYS WITH RANDOM VALUES

```
int[][] matrix = new int[3][3];
```

```
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    matrix[row][column] = (int)(Math.random() * 100);
  }
}
```



PRINTING ARRAYS

```
int[][] matrix = new int[3][3];
```

```
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    System.out.print(matrix[row][column] + " ");
  }

  System.out.println();
}
```

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];
```

row	column	total
		0

```
int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

row	column	total
0	0	0

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150
	2	

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150
	2	210

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150
	2	210
2	0	

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150
	2	210
2	0	280

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150
	2	210
2	0	280
	1	

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150
	2	210
2	0	280
	1	360

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150
	2	210
2	0	280
	1	360
	2	

SUMMING ALL ELEMENTS

```
int[][] matrix = new int[3][3];

int total = 0;
for (int row = 0; row < matrix.length; row++) {
  for (int column = 0; column < matrix[row].length; column++) {
    total += matrix[row][column];
  }
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

row	column	total
0	0	0
		10
	1	30
	2	60
1	0	100
	1	150
	2	210
2	0	280
	1	360
	2	450

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
	0	

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50
2		

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50
2		120

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50
2		120

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50
2		120
	1	0

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		150

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120
Sum for column 1 is 150

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		150

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120
Sum for column 1 is 150

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		150
	2	0

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120
Sum for column 1 is 150

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		150
	2	0
0		30

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120
Sum for column 1 is 150

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		150
	2	0
0		30
1		

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120
Sum for column 1 is 150

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		150
	2	0
0		30
1		90

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120
Sum for column 1 is 150

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		150
	2	0
0		30
1		90
2		

SUMMING ELEMENTS BY COLUMN

```
int[][] matrix = new int[3][3];

for (int column = 0; column < matrix[0].length; column++) {
  int total = 0;
  for (int row = 0; row < matrix.length; row++)
    total += matrix[row][column];
  System.out.println("Sum for column " + column + " is "
    + total);
}
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

Sum for column 0 is 120
Sum for column 1 is 150
Sum for column 2 is 180

row	column	total
0	0	0
		10
1		50
2		120
	1	0
0		20
1		70
2		150
	2	0
0		30
1		90
2		180

```
int maxRow = 0;
int indexOfMaxRow = 0;

// Get sum of the first row in maxRow
for (int column = 0; column < matrix[0].length; column++) {
  maxRow += matrix[0][column];
}

for (int row = 1; row < matrix.length; row++) {
  int totalOfThisRow = 0;
  for (int column = 0; column < matrix[row].length; column++)
    totalOfThisRow += matrix[row][column];

  if (totalOfThisRow > maxRow) {
    maxRow = totalOfThisRow;
    indexOfMaxRow = row;
  }
}

System.out.println("Row " + indexOfMaxRow
  + " has the maximum sum of " + maxRow);
```

ROW WITH THE LARGEST SUM

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90

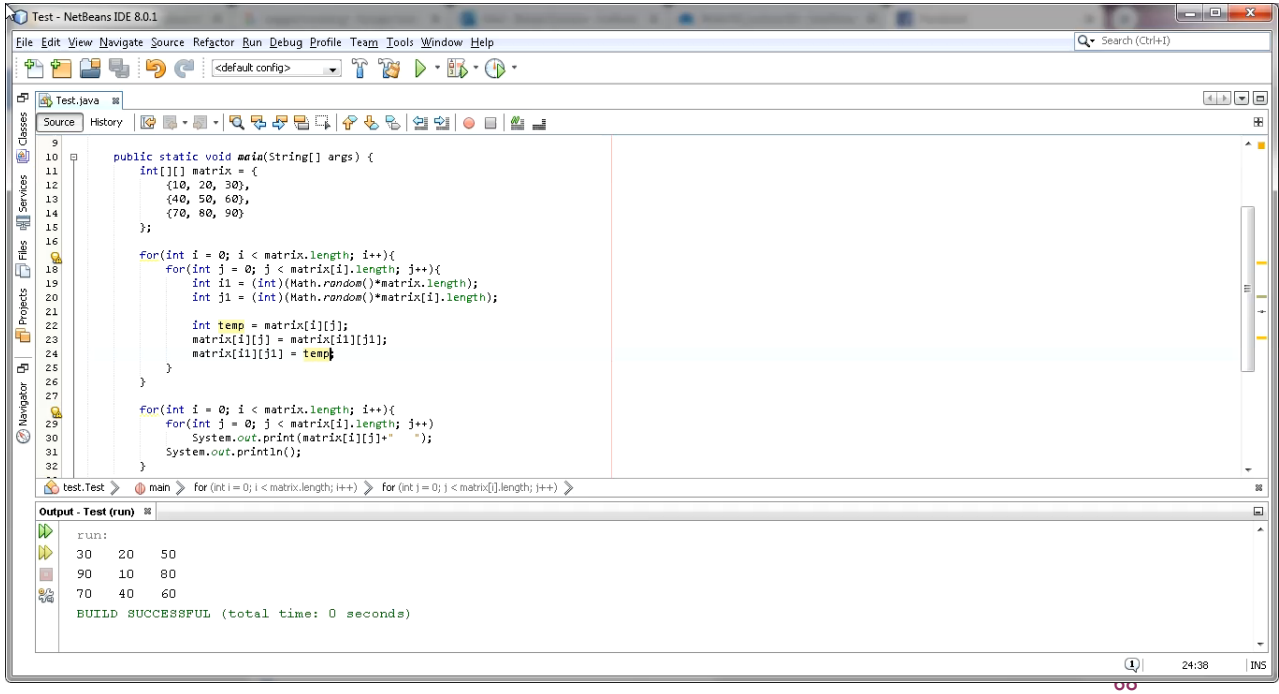
RANDOM SHUFFLING

```
for (int i = 0; i < matrix.length; i++) {
  for (int j = 0; j < matrix[i].length; j++) {
    int i1 = (int)(Math.random() * matrix.length);
    int j1 = (int)(Math.random() * matrix[i].length);

    // Swap matrix[i][j] with matrix[i1][j1]

      int temp = matrix[i][j];
      matrix[i][j] = matrix[i1][j1];
      matrix[i1][j1] = temp;
    }
 }
```

	[0]	[1]	[2]
[0]	10	20	30
[1]	40	50	60
[2]	70	80	90



EXAMPLE CODES

```
int[][] array = {{1, 2}, {3, 4}, {5, 6}};
for (int i = array.length - 1; i >= 0; i--) {
  for (int j = array[i].length - 1; j >= 0; j--)
    System.out.print(array[i][j] + " ");
  System.out.println();
}
```

```
6 5
4 3
2 1
```

```
int[][] array = {{1, 2}, {3, 4}, {5, 6}};
int sum = 0;
for (int i = 0; i < array.length; i++)
  sum += array[i][0];
System.out.println(sum);
```

```
9
```

PASSING/RETURNING 2D ARRAYS TO/FROM METHODS

- When passing a two-dimensional array to a method, the reference of the array is passed to the method

```
Enter 3 rows and 4 columns:
1 2 3 4 Enter
5 6 7 8 Enter
9 10 11 12 Enter

Sum of all elements is 78
```

```
1  import java.util.Scanner;
2
3  public class PassTwoDimensionalArray {
4    public static void main(String[] args) {
5      int[][] m = getArray(); // Get an array
6
7      // Display sum of elements
8      System.out.println("\nSum of all elements is " + sum(m));
9    }
10
11   public static int[][] getArray() {
12     // Create a Scanner
13     Scanner input = new Scanner(System.in);
14
15     // Enter array values
16     int[][] m = new int[3][4];
17     System.out.println("Enter " + m.length + " rows and "
18       + m[0].length + " columns: ");
19     for (int i = 0; i < m.length; i++)
20       for (int j = 0; j < m[i].length; j++)
21         m[i][j] = input.nextInt();
22
23     return m;
24   }
25
26   public static int sum(int[][] m) {
27     int total = 0;
28     for (int row = 0; row < m.length; row++) {
29       for (int column = 0; column < m[row].length; column++) {
30         total += m[row][column];
31       }
32     }
33
34     return total;
35   }
36 }
```

EXAMPLE CODE

```
public class Test {
  public static void main(String[] args) {
    int[][] array = {{1, 2, 3, 4}, {5, 6, 7, 8}};
    System.out.println(m1(array)[0]);
    System.out.println(m1(array)[1]);
  }

  public static int[] m1(int[][] m) {
    int[] result = new int[2];
    result[0] = m.length;
    result[1] = m[0].length;
    return result;
  }
}
```

Output:

2
4

GRADING A MCQ TEST

- **Problem**: write a program that will grade multiple-choice tests.
- **Description**: suppose you need to write a program that grades MCQs. Assume there are 10 students and 5 questions. You also have the key to correct answers.

	Q1	Q2	Q3	Q4	Q5
Student 1	A	B	A	C	C
Student 2	D	B	A	B	C
Student 3	E	D	D	A	C
Student 4	A	B	D	C	D
Student 5	B	B	E	C	C
Student 6	B	B	A	C	C
Student 7	E	B	E	C	C
Student 8	E	A	D	E	A
Student 9	C	E	E	B	A
Student 10	A	B	A	C	C

```
char[][] answers = {

      {'A','B','A','C','C'},
       {'D','B','A','B','C'},
       {'E','D','D','A','C'},
      {'A','B','D','C','D'},
      {'B','B','E','C','C'},
      {'B','B','A','C','C'},
      {'E','B','E','C','C'},
      {'E','A','D','E','A'},
      {'C','E','E','B','A'},
      {'A','B','A','C','C'}

};
```

	Q1	Q2	Q3	Q4	Q5
Correct answer	D	B	D	C	C

```
char[] keys = {'D', 'B', 'D', 'C', 'C'};
```

CODE

LISTING 8.2 GradeExam.java

```
1  public class GradeExam {
2    /** Main method */
3    public static void main(String[] args) {
4      // Students' answers to the questions
5      char[][] answers = {                                          2-D array
6        {'A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
7        {'D', 'B', 'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'},
8        {'E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A', 'D'},
9        {'C', 'B', 'A', 'E', 'D', 'C', 'E', 'E', 'A', 'D'},
10       {'A', 'B', 'D', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
11       {'B', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
12       {'B', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
13       {'E', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'}};
14
15     // Key to the questions
16     char[] keys = {'D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A', 'D'};   1-D array
17
18     // Grade all answers
19     for (int i = 0; i < answers.length; i++) {
20       // Grade one student
21       int correctCount = 0;
22       for (int j = 0; j < answers[i].length; j++) {
23         if (answers[i][j] == keys[j])                              compare with key
24           correctCount++;
25       }
26
27       System.out.println("Student " + i + "'s correct count is " +
28         correctCount);
29     }
30   }
31 }
```

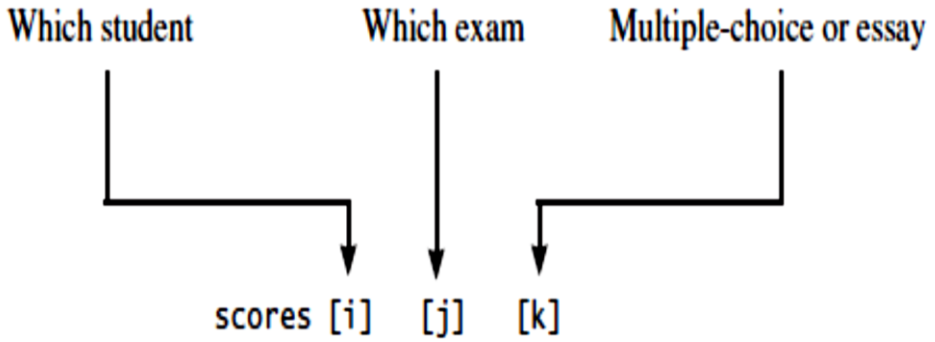
MULTIDIMENSIONAL ARRAYS

- A 2D array consists of a collection (or an array) of 1D arrays
- A 3D array consists of an array of 2D arrays
  - Ex: A 3D array to store exam scores for a class of six students with five courses, and each course has two parts (lab and theory)

```
double[][][] scores = new double[6][5][2];
```

- n-dimensional arrays for any integer n

```
double[][][] scores = {
  {{7.5, 20.5}, {9.0, 22.5}, {15, 33.5}, {13, 21.5}, {15, 2.5}},
  {{4.5, 21.5}, {9.0, 22.5}, {15, 34.5}, {12, 20.5}, {14, 9.5}},
  {{6.5, 30.5}, {9.4, 10.5}, {11, 33.5}, {11, 23.5}, {10, 2.5}},
  {{6.5, 23.5}, {9.4, 32.5}, {13, 34.5}, {11, 20.5}, {16, 7.5}},
  {{8.5, 26.5}, {9.4, 52.5}, {13, 36.5}, {13, 24.5}, {16, 2.5}},
  {{9.5, 20.5}, {9.4, 42.5}, {13, 31.5}, {12, 20.5}, {16, 6.5}}};
```



`scores[0][1][0]` refers to the lab score for the first student's second course

`scores[0][1][0]` refers to the lab score for the first student's second course

```
scores.length is 6
scores[0].length is 5
scores[0][0].length is 2
```